

Programming in Java

(BCAC 0012)

Module-3

Section – 2

Multithreaded Programming

Faculty Name – Dr. Vinod Jain

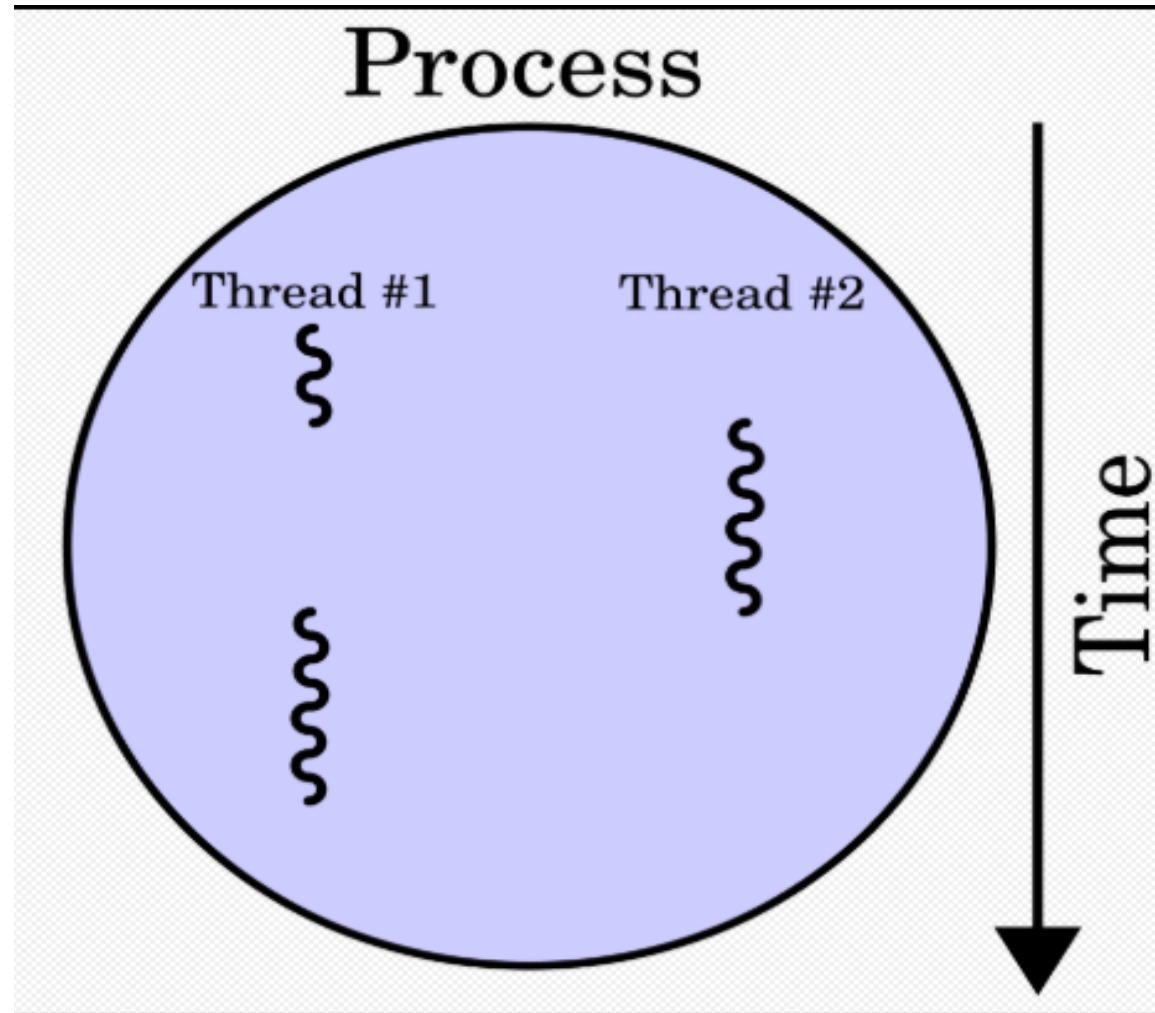
Syllabus

3	Exception Handling	Introduction to Exception, Type of Exceptions
		Keywords-Try ,catch, Throw, Throws, Finally
		Try-catch Block ,Multiple Catch Clauses, Nested Try Statements
		Java's Built-in Exceptions
		Creating Your Own Exception Subclasses.
	Multithreaded Programming	Thread Life Cycle
		Creating Threads , Thread Methods
		Thread Priority,
	Introduction to Applet Programming	Introduction to Applet Programming, Difference between Applet and application
		Graphics class methods and implementation
		Applet Life Cycle and its methods
		Applet tag
		Use of Param tag, Applet Programming

Module 3 Section 2 Multithreaded Programming

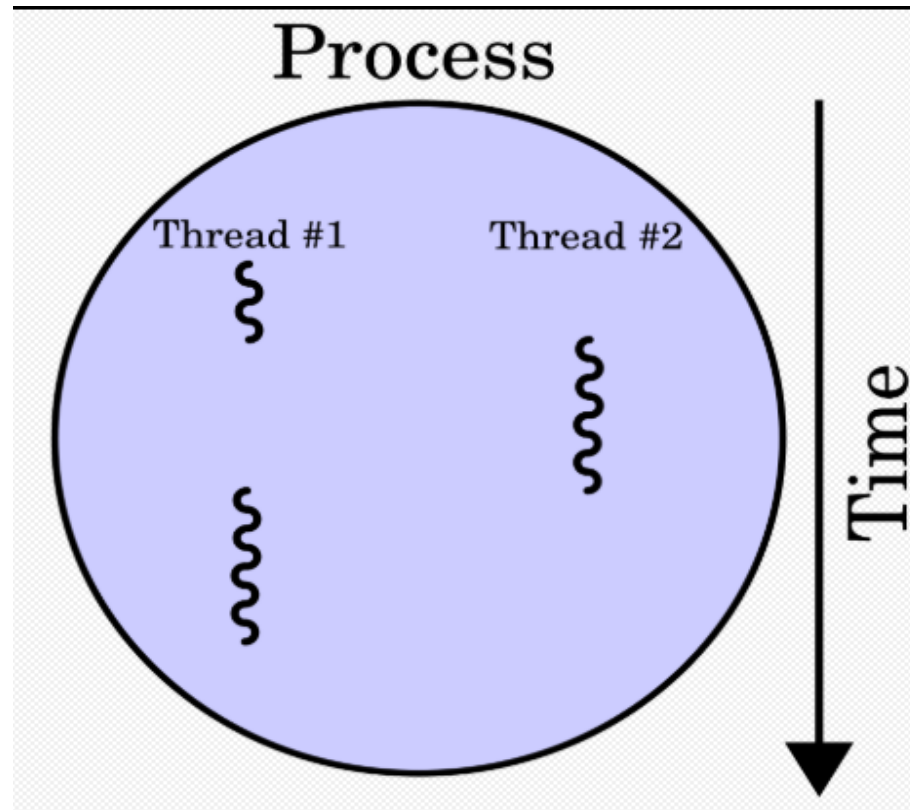
- Thread Basic Concepts
- Thread Life Cycle
- Creating Threads
- Thread Methods
- Thread Priority

Thread Basic Concept



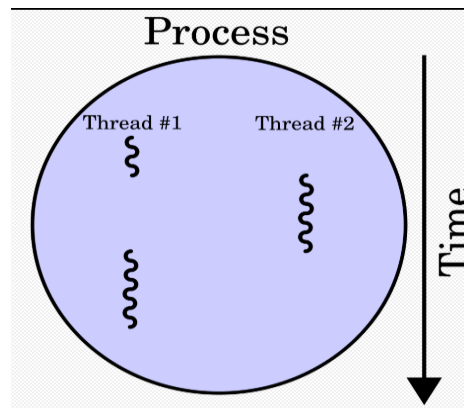
What is Thread

- A thread is a lightweight sub-process.



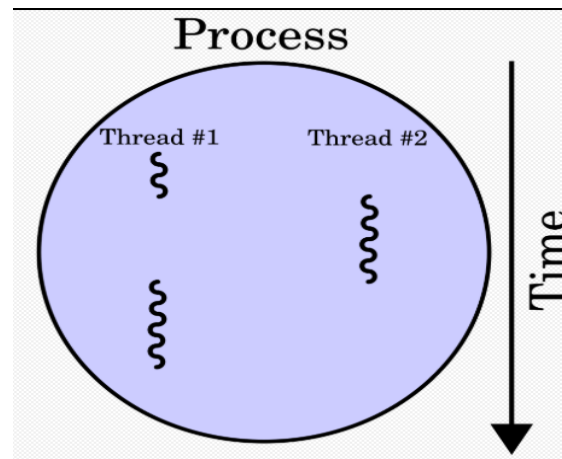
What is Thread

1. A thread is a lightweight subprocess, the smallest unit of processing.
2. It is a separate path of execution.
3. Threads are independent.
4. If there occurs exception in one thread, it doesn't affect other threads.

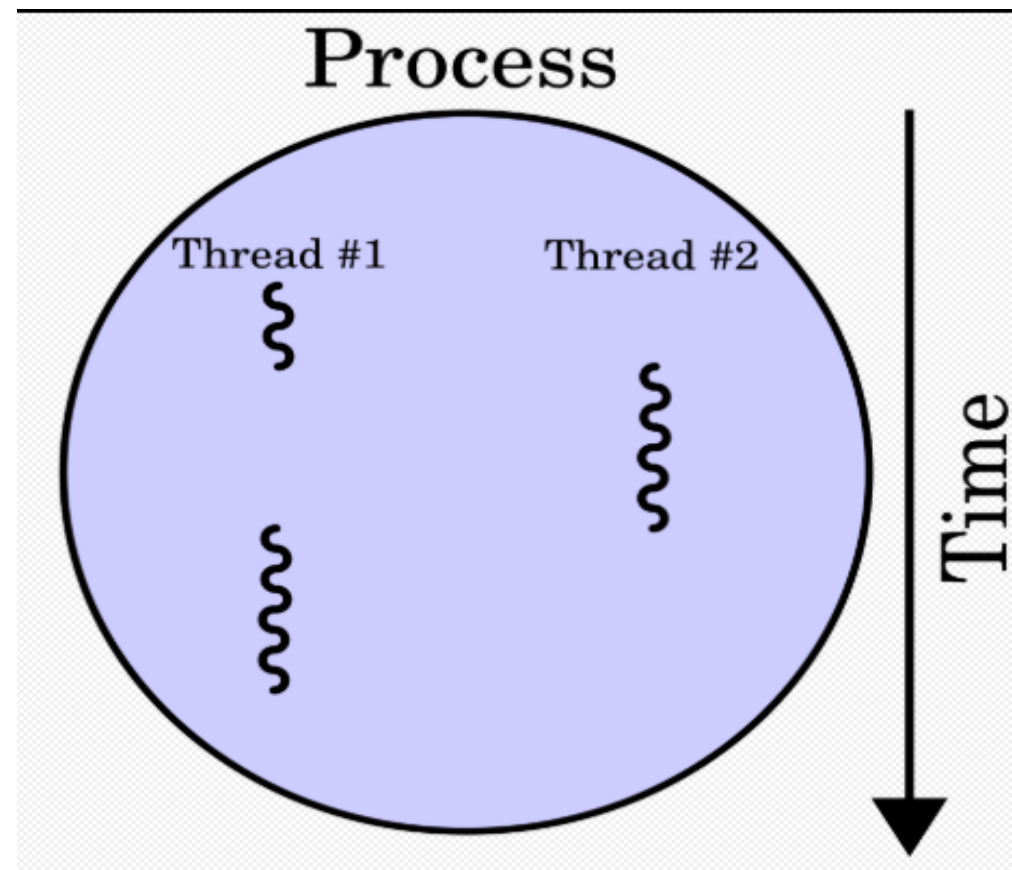
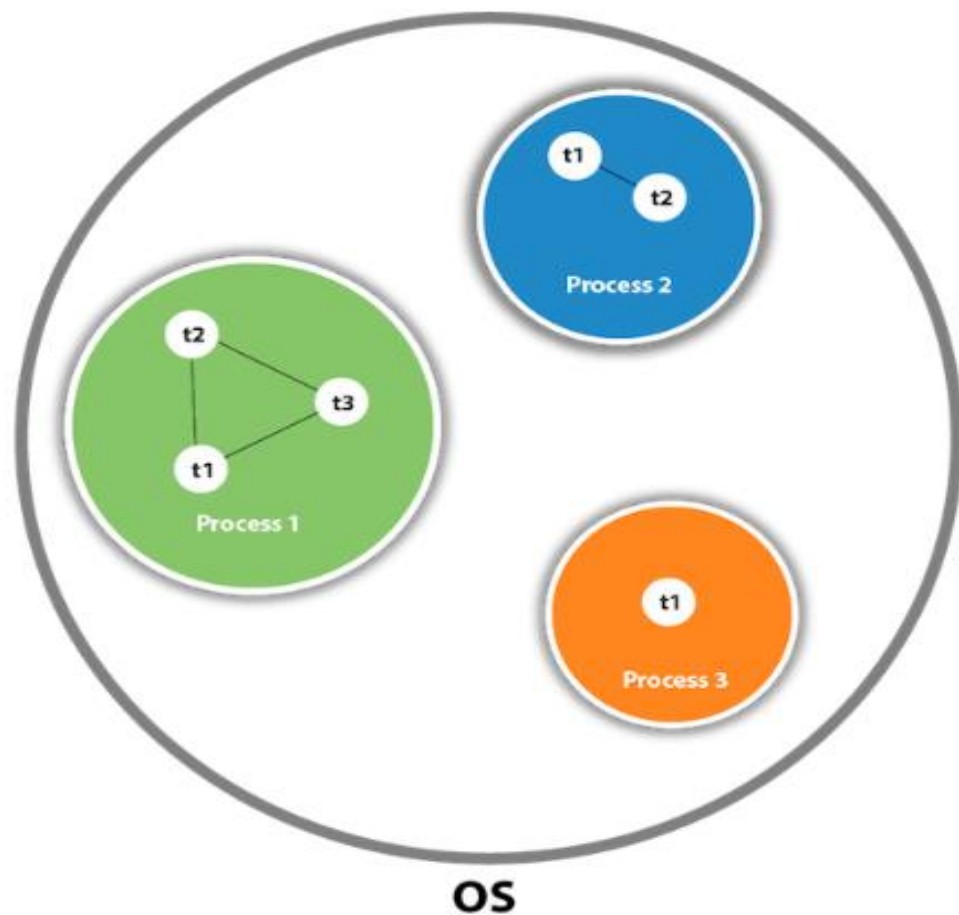


What is Thread

1. It uses a shared memory area.
2. A thread is executed inside the process.
3. There is context-switching between the threads. There can be multiple processes inside the OS, and one process can have multiple threads.



Multitasking with Multi-threading



Multi-threading

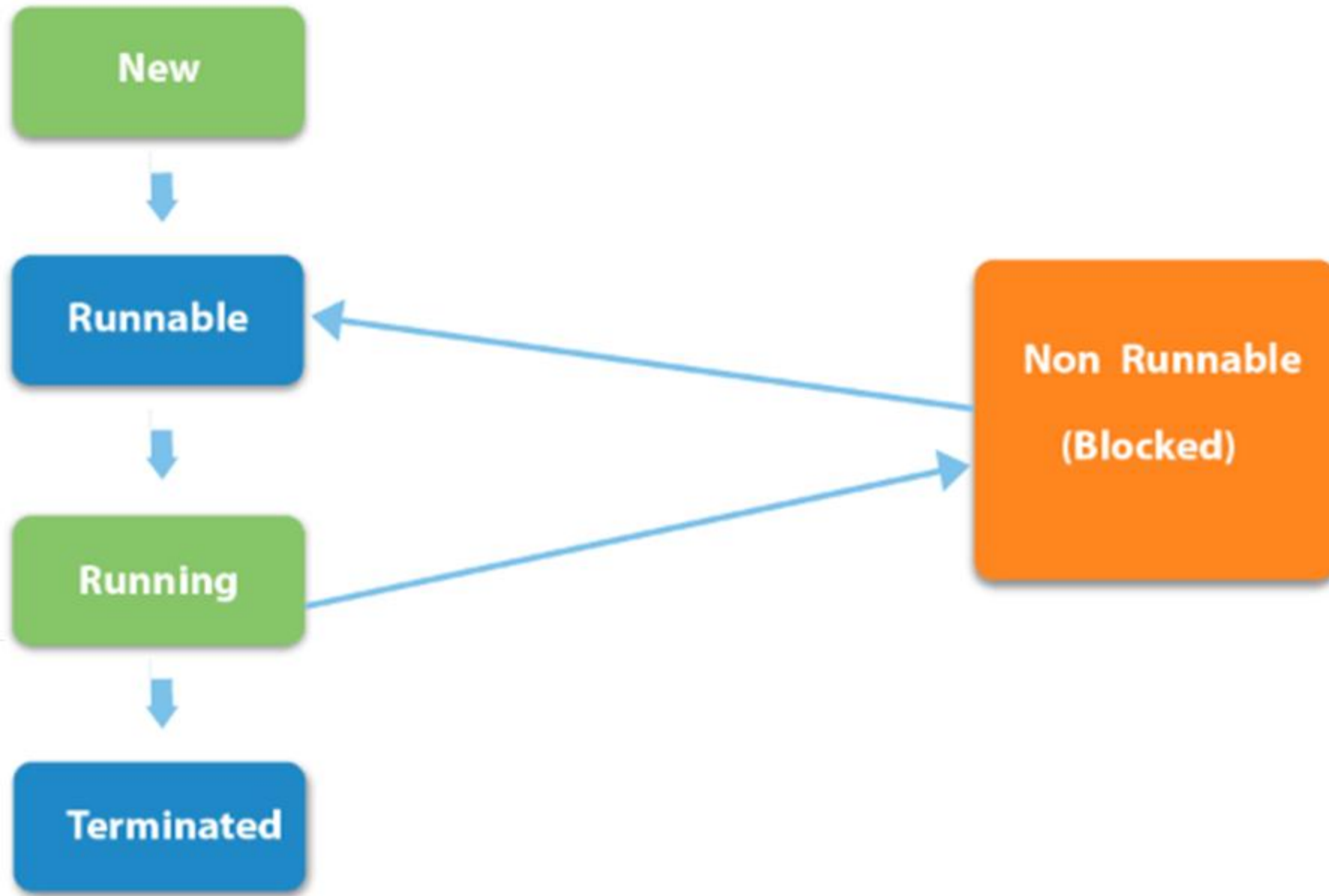
- Multi-threading extends the idea of multitasking into applications where you can subdivide specific operations within a single application into individual threads.
- Each of the threads can run in parallel.
- The OS divides processing time not only among different applications, but also among each thread within an application.

Advantages of Java Multithreading

- 1) It doesn't block the user because threads are independent and you can perform multiple operations at the same time.
- 2) You can perform many operations together, so it saves time.
- 3) Threads are independent, so it doesn't affect other threads if an exception occurs in a single thread.
- 4) Some background low priority threads can be created that performs less important activities like taking backups etc.

Life Cycle of a Thread

- Thread undergoes through different states in its lifecycle. These states are :
 1. New
 2. Runnable
 3. Running
 4. Non-Runnable (Blocked)
 5. Terminated



Life Cycle of a Thread

- A thread can be in one of the five states.
- According to sun, there is **only 4 states** in thread life cycle in java new, runnable, non-runnable and terminated. **There is no running state.**
- But for better understanding the threads, we are explaining it in the 5 states, Running state is added for better understanding.
- The life cycle of the thread in java is controlled by JVM. The java thread states are as follows:

States diagram of thread lifecycle

States of thread lifecycle

1) New

The thread is in new state if you create an instance of Thread class but before the invocation of start() method.

2) Runnable

The thread is in runnable state after invocation of start() method, but the thread scheduler has not selected it to be the running thread.

3) Running

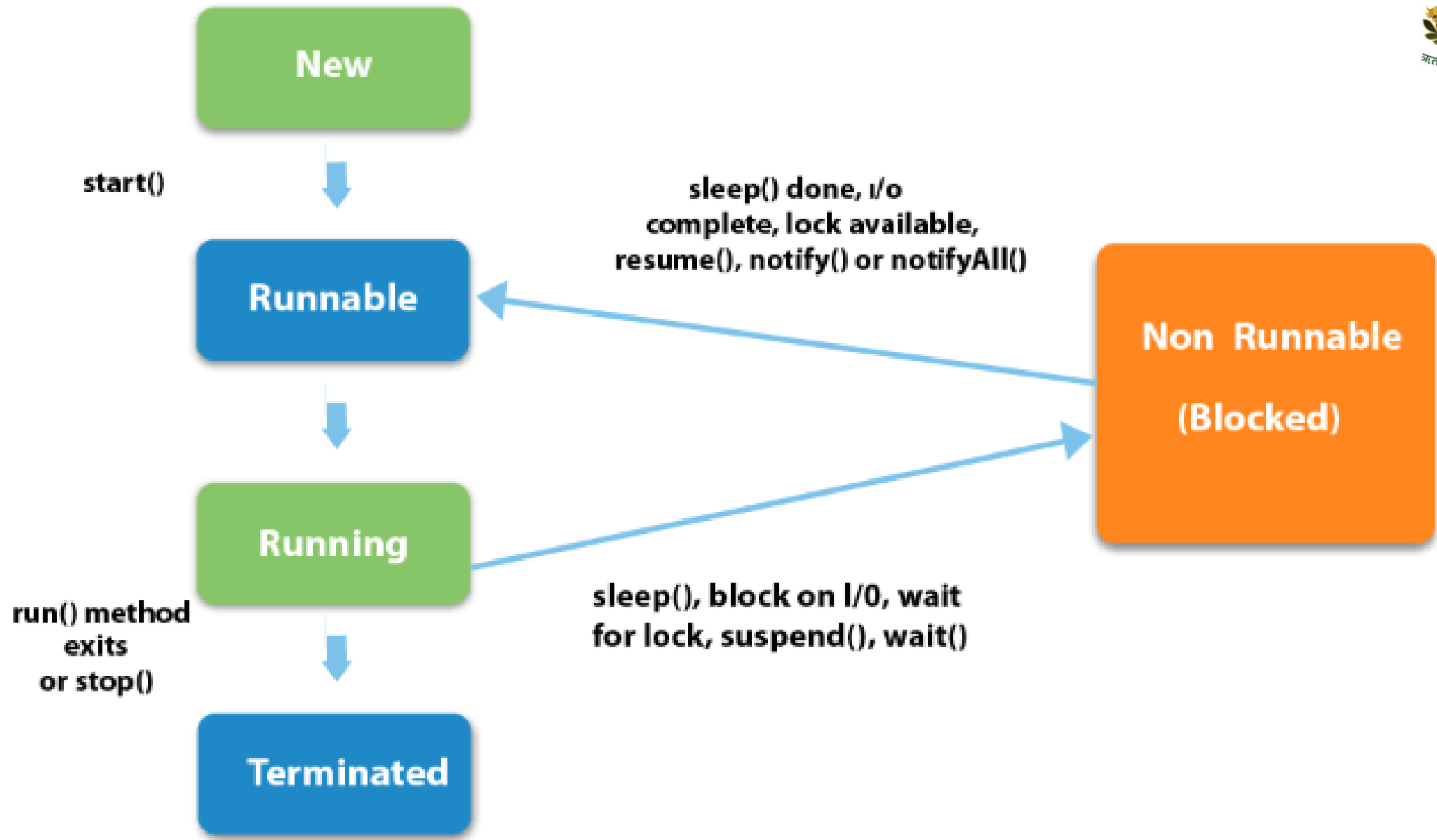
The thread is in running state if the thread scheduler has selected it.

4) Non-Runnable (Blocked)

This is the state when the thread is still alive, but is currently not eligible to run.

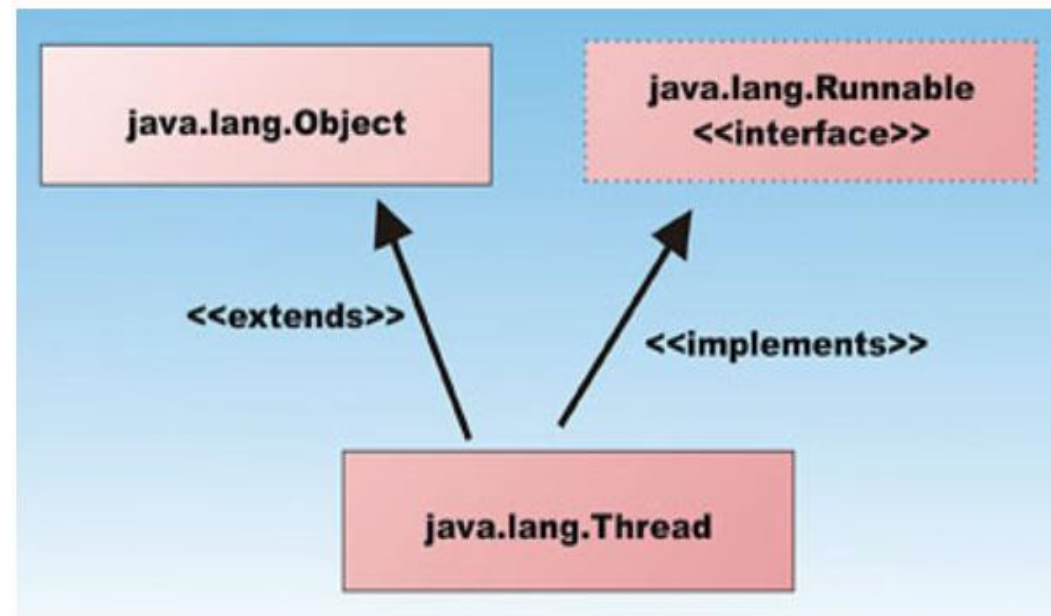
5) Terminated

A thread is in terminated or dead state when its run() method exits.



Creating Threads : Thread class:

- Thread class provide constructors and methods to create and perform operations on a thread.
- Thread class extends Object class and implements Runnable interface.



Commonly used Constructors of Thread class:

- Thread()
- Thread(String name)
- Thread(Runnable r)
- Thread(Runnable r,String name)

Commonly used methods of Thread class:

1. public void **run**(): is used to perform action for a thread.
2. public void **start**(): starts the execution of the thread. JVM calls the run() method on the thread.
3. public void **sleep**(long milliseconds): Causes the currently executing thread to sleep (temporarily cease execution) for the specified number of milliseconds.
4. public void **join**(): waits for a thread to die.
5. public void **join** (long milliseconds): waits for a thread to die for the specified milliseconds.
6. public int **getPriority**(): returns the priority of the thread.
7. public int **setPriority**(int priority): changes the priority of the thread.
8. public String **getName**(): returns the name of the thread.

Commonly used methods of Thread class:

9. public void **setName** (String name): changes the name of the thread.
10. public Thread **currentThread**(): returns the reference of currently executing thread.
11. public int **getId**(): returns the id of the thread.
12. public Thread.State **getState**(): returns the state of the thread.
13. public boolean **isAlive**(): tests if the thread is alive.
14. public void **yield**(): causes the currently executing thread object to temporarily pause and allow other threads to execute.
15. public void **suspend**(): is used to suspend the thread(deprecated).
16. public void **resume**(): is used to resume the suspended thread(deprecated).

Commonly used methods of Thread class:

- 17. public void **stop()**: is used to stop the thread(deprecated).
- 18. public boolean **isDaemon()**: tests if the thread is a daemon thread.
- 19. public void **setDaemon(boolean b)**: marks the thread as daemon or user thread.
- 20. public void **interrupt()**: interrupts the thread.
- 21. public boolean **isInterrupted()**: tests if the thread has been interrupted.
- 22. public static boolean **interrupted()**: tests if the current thread has been interrupted.

How to create thread

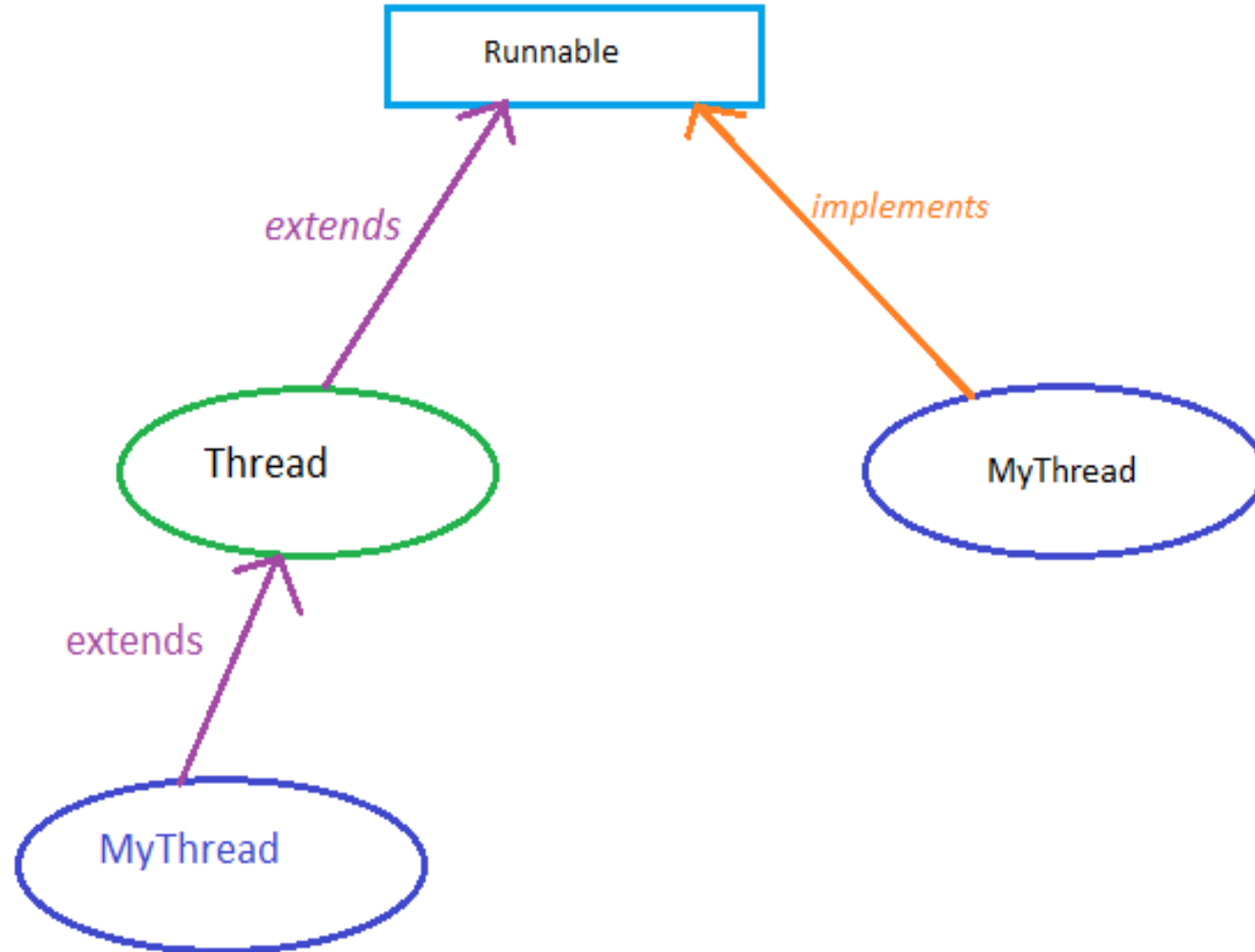


fig- creating MyThread(User Defined Thread)

How to create thread

- There are two ways to create a thread:
 1. By extending Thread class
 2. By implementing Runnable interface

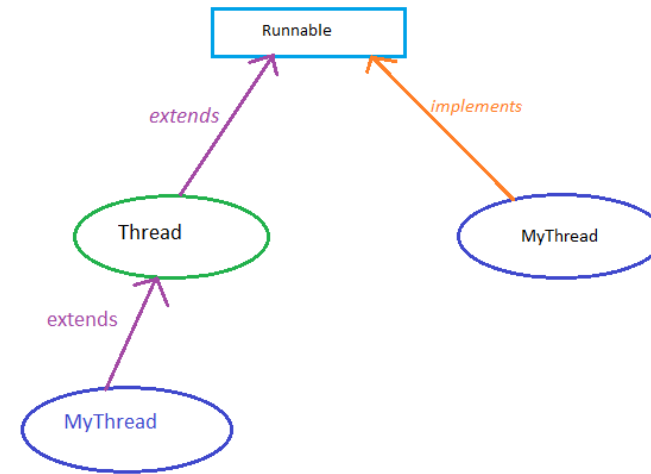


fig- creating MyThread(User Defined Thread)

By extending Thread class

The first way to create a thread is to create a new class that extends Thread class using the following two simple steps.

Step 1

You will need to override run() method available in Thread class. This method provides an entry point for the thread. Following is a simple syntax of run() method

```
public void run( )
```

Step 2

Once Thread object is created, you can start it by calling start() method, which executes a call to run() method. Following is a simple syntax of start() method

```
void start( );
```


Step 1 : override run() method

```
class A extends Thread
```

```
{
```

```
//overriding run() method
```

```
public void run()
```

```
{
```

```
    for(int i=1;i<=10;i++)
```

```
    {
```

```
        System.out.print("\n Thread A : i = " + i);
```

```
    }
```

```
}
```

```
}
```

Step 2 : calling start() method

```
class ThreadDemo
{
    public static void main(String[] args)
    {
        //start the thread by calling start() method.
        A a = new A();
        a.start();
    }
}
```



// Implementing Multithreading using Thread Class

```
package ThreadPackage;
class A extends Thread
{
    //overriding run() method
    public void run()
    {
        for(int i=1;i<=10;i++)
        {
            System.out.print("\n Thread A : i = " + i);
        }
    }
}
class B extends Thread
{
    //overriding run() method
    public void run()
    {
        for(int i=1;i<=10;i++)
        {
            System.out.print("\n Thread B : i = " + i);
        }
    }
}
```

```
class C extends Thread
{
    //overriding run() method
    public void run()
    {
        for(int i=1;i<=10;i++)
        {
            System.out.print("\n Thread C : i = " + i);
        }
    }
}
class ThreadDemo
{
    public static void main(String[] args) {
        //start the thread by calling start() method.
        A a = new A();
        B b = new B();
        C c = new C();
        a.start();
        b.start();
        c.start();
    }
}
```

Output

Thread A : i = 1
Thread C : i = 1
Thread A : i = 2
Thread B : i = 1
Thread A : i = 3
Thread C : i = 2
Thread C : i = 3
Thread C : i = 4
Thread C : i = 5
Thread C : i = 6
Thread C : i = 7

Thread C : i = 8
Thread C : i = 9
Thread C : i = 10
Thread A : i = 4
Thread A : i = 5
Thread A : i = 6
Thread A : i = 7
Thread A : i = 8
Thread A : i = 9
Thread A : i = 10
Thread B : i = 2

Thread B : i = 3
Thread B : i = 4
Thread B : i = 5
Thread B : i = 6
Thread B : i = 7
Thread B : i = 8
Thread B : i = 9
Thread B : i = 10
BUILD
SUCCESSFUL (total time: 0
seconds)

Code

// Implementing Multithreading using Thread Class

```
package ThreadPackage;
```

```
class A extends Thread
```

```
{
```

```
    //overriding run() method
```

```
        public void run()
```

```
        {
```

```
            for(int i=1;i<=10;i++)
```

```
            {
```

```
                System.out.print("\n Thread A : i = " + i);
```

```
            }
```

```
        }
```

```
}
```

```
class B extends Thread
```

```
{
```

```
    //overriding run() method
```

```
        public void run()
```

```
{
```

```
    for(int i=1;i<=10;i++)
```

```
    {
```

```
        System.out.print("\n Thread B : i = " + i);
```

```
    }
```

```
}
```

```
}
```

```
class C extends Thread
```

```
{
```

```
    //overriding run() method
```

```
        public void run()
```

```
        {
```

```
            for(int i=1;i<=10;i++)
```

```
            {
```

```
                System.out.print("\n Thread C : i = " + i);
```

```
class ThreadDemo
```

```
{
```

```
    public static void main(String[] args) {
```

```
        start() method.
```

```
        //start the thread by calling
```

```
        A a = new A();
```

```
        B b = new B();
```

```
        C c = new C();
```

```
        a.start();
```

```
        b.start();
```

```
        c.start();
```

```
    }
```

```
}
```

By implementing Runnable interface.

1. The Runnable interface should be implemented by any class whose instances are intended to be executed by a thread.
2. Runnable interface have only one method named run().
3. The run method has following header
public void run():
4. It is used to perform action for a thread.

```
// Implementing Multithreading using Runnable interface
package ThreadPackage;
class D implements Runnable
{
    //overriding run() method
    public void run()
    {
        for(int i=1;i<=10;i++)
        {
            System.out.print("\n Thread D : i = " + i);
        }
    }
}

public class ThreadusingRunnable1
{
    public static void main(String[] args) {

        //start the thread by calling start() method.
        D d = new D();
        d.run();
    }
}
```

```
// Implementing Multithreading using Runnable interface
package ThreadPackage;
class D implements Runnable
{
    //overriding run() method
    public void run()
    {
        for(int i=1;i<=10;i++)
        {
            System.out.print("\n Thread D : i = " + i);
        }
    }
}

public class ThreadusingRunnable1
{
    public static void main(String[] args) {

        //start the thread by calling start() method.
        D d = new D();
        d.run();
    }
}
```

Output

run:

```
Thread D : i = 1
Thread D : i = 2
Thread D : i = 3
Thread D : i = 4
Thread D : i = 5
Thread D : i = 6
Thread D : i = 7
Thread D : i = 8
Thread D : i = 9
Thread D : i = 10BUILD SUCC
```


Code

```
• // Implementing Multithreading using Runnable interface
• package ThreadPackage;
• class D implements Runnable
• {
•     //overriding run() method
•     public void run()
•     {
•         for(int i=1;i<=10;i++)
•         {
•             System.out.print("\n Thread D : i = " + i);
•         }
•     }
• }
• public class ThreadusingRunnable1
• {
•     public static void main(String[] args) {
•
•         //start the thread by calling start() method.
•         D d = new D();
•         d.run();
•     }
• }
```

Priority of a Thread (Thread Priority):

- Each thread have a priority.
- Priorities are represented by a number between 1 and 10.
- It helps the operating system determine the order in which threads are scheduled.
- Java thread priorities are in the range between MIN_PRIORITY (a constant of 1) and MAX_PRIORITY (a constant of 10).
- By default, every thread is given priority NORM_PRIORITY (a constant of 5).

Priority of a Thread (Thread Priority):

- Threads with higher priority are more important to a program and should be allocated processor time before lower-priority threads.
- In most cases, thread scheduler schedules the threads according to their priority (known as preemptive scheduling).
- But it is not guaranteed because it depends on JVM specification that which scheduling it chooses.
-

Priority of a Thread (Thread Priority):

3 constants defined in Thread class are :

1. `public static int MIN_PRIORITY`
2. `public static int NORM_PRIORITY`
3. `public static int MAX_PRIORITY`

MIN_PRIORITY (a constant of 1)

NORM_PRIORITY (a constant of 5)

MAX_PRIORITY (a constant of 10).

```
// Program to implement Thread Priority
package ThreadPackage;
class F extends Thread
{
    //overriding run() method
    public void run()
    {
        for(int i=1;i<=10;i++)
        {
            System.out.print("\n Thread F : i = " + i);
        }
    }
}
class G extends Thread
{
    //overriding run() method
    public void run()
    {
        for(int i=1;i<=10;i++)
        {
            System.out.print("\n Thread G : i = " + i);
        }
    }
}
```

```
public class ThreadPriority1
{
    public static void main(String[] args) {
        //start the thread by calling start() method.
        F f = new F();
        G g = new G();
        f.setPriority(Thread.MIN_PRIORITY);
        g.setPriority(Thread.MAX_PRIORITY);
        f.start();
        g.start();
    }
}
```

```
// Program to implement Thread Priority
package ThreadPackage;
class F extends Thread
{
    //overriding run() method
    public void run()
    {
        for(int i=1;i<=10;i++)
        {
            System.out.print("\n Thread F : i = " + i);
        }
    }
}
class G extends Thread
{
    //overriding run() method
    public void run()
    {
        for(int i=1;i<=10;i++)
        {
            System.out.print("\n Thread G : i = " + i);
        }
    }
}
```

```
public class ThreadPriority1
{
    public static void main(String[] args) {
        //start the thread by calling start() method.
        F f = new F();
        G g = new G();
        f.setPriority(Thread.MIN_PRIORITY);
        g.setPriority(Thread.MAX_PRIORITY);
        f.start();
        g.start();
    }
}
```

```
Output :
run:
Thread G : i = 1
Thread G : i = 2
Thread G : i = 3
Thread G : i = 4
Thread G : i = 5
Thread G : i = 6
Thread G : i = 7
Thread G : i = 8
Thread F : i = 1
Thread G : i = 9
Thread G : i = 10
Thread F : i = 2
Thread F : i = 3
Thread F : i = 4
Thread F : i = 5
Thread F : i = 6
Thread F : i = 7
Thread F : i = 8
Thread F : i = 9
Thread F : i = 10
```

Code

```
// Program to implement Thread Priority
package ThreadPackage;
class F extends Thread
{
    //overriding run() method
    public void run()
    {
        for(int i=1;i<=10;i++) = " + i);
        System.out.print("\n Thread F : i
        = " + i);
    }
}

class G extends Thread
{
    //overriding run() method
    public void run()
    {
        for(int i=1;i<=10;i++)
        {
            System.out.print("\n Thread G : i
            = " + i);
        }
    }
}

public class ThreadPriority1
{
    public static void main(String[] args) {
        //start the thread
        //by calling start() method.
        F f = new F();
        G g = new G();
        f.setPriority(Thread.MIN_PRIORITY);
        g.setPriority(Thread.MAX_PRIORITY);
        f.start();
        g.start();
    }
}
```